MODULE 22 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

UDFs, procedures, Snowpark, and the Domain 4 drill

The last of your own code, then six scenarios across modules 17 to 22.

DOMAIN 4 of 5 WEIGHT 21% MODULE 22 of 31

admin@sthithapragna.com

UDF or stored procedure?

USER-DEFINED FUNCTION

Returns a value into a query

- Always returns a value, explicitly
- Called as part of a SQL expression
- Many can appear in one statement
- The return value is usable directly in SQL

STORED PROCEDURE

Performs operations

- May or may not return a value
- Called with CALL, as its own statement
- One per CALL statement
- CALL provides nowhere to put the return value

A DETAIL WORTH HAVING

Every CREATE PROCEDURE needs a RETURNS clause even if it returns nothing. Without an explicit return it gives NULL.

Getting a procedure's value back anyway

- You **cannot** write x = my_procedure(y). CALL has nowhere to put it.
- Inside a **Snowflake Scripting** block, you can capture the return value in a variable.
- Call the procedure **inside another procedure**, where the handler language can hold the result.
- A procedure returning **tabular data** can be called in the **FROM clause** of a SELECT.
- Call it, then use **RESULT_SCAN** with its statement ID. The same function from module 18.

> ON THE EXAM
>
> Four documented workarounds. If a scenario needs the value inline in ordinary SQL, though, the answer is a UDF.

Owner's rights and caller's rights

- **Owner's rights is the default.** The procedure runs with the **owner's** privileges.
- That is how an owner **delegates an operation** to users who could not otherwise perform it.
- **Owner's rights procedures cannot change session state.** No setting session variables or parameters.
- **Caller's rights** runs with the caller's privileges, and inherits the caller's warehouse, database and schema.
- A caller's rights procedure **can view, set and unset session variables**, and those changes **persist after the CALL**.

> ON THE EXAM
>
> Owner's rights is the security answer: give somebody the ability to do one specific thing without granting the underlying privilege.

Languages and UDF variants

WHAT	DETAIL
Handler languages	SQL, JavaScript, Python, Java and Scala. The same set for both UDFs and procedures.
Scalar UDF	Returns one value per input row. The default kind.
UDTF	A table function. Returns a set of rows rather than one value.
Secure UDF	Hides the definition. Required if the function is to be shared.
Vectorized Python UDF	Processes batches of rows rather than one at a time. Faster for numeric work.
Snowflake Scripting UDF	A SQL UDF containing procedural code. Returns a scalar, and is callable in a SELECT.

ON THE EXAM

UDTF is the one to remember. If a scenario needs a function to return several rows per input, a scalar UDF cannot do it.

Snowpark, in one slide

- A **DataFrame API** for Python, Java and Scala. Operations are **lazy** and translate into SQL that runs on a warehouse.
- The point is **pushdown**. Work happens in the engine rather than pulling rows to a client.
- It is also how you author **UDFs, UDTFs and stored procedures** in those languages.
- **Snowpark-optimized warehouses** give more memory per node. Standard warehouse memory does not grow with size.

> ON THE EXAM
>
> Module 6 covered this properly. The line worth carrying is that a Snowpark DataFrame chain appears as ordinary SQL in Query History.

> A team has written a stored procedure that calculates a risk score from several inputs. Analysts now want to use that score inside ordinary queries, as in SELECT customer_id, risk_score(...) FROM customers.
>
> The statement fails. An engineer suggests wrapping the procedure in a view.

What is the correct approach?

A Rewrite the logic as a UDF. A UDF returns a value into a SQL expression; a procedure cannot.

B Grant USAGE on the procedure to the analyst role, which allows inline invocation.

C Wrap the procedure in a view as suggested. Views can call procedures per row.

D Change the procedure to caller's rights, which permits use inside a SELECT.

◆ PAUSE AND DECIDE

What is the correct approach?

A Rewrite it as a UDF.

A UDF is called as part of a SQL expression and returns its value directly. That is precisely what the analysts need.

B Grant USAGE on the procedure.

USAGE lets a role call the procedure. It does not change how a procedure may be invoked.

C Wrap it in a view.

A view is a stored query. It cannot invoke a procedure per row, so the restriction is unchanged.

D Switch to caller's rights.

The rights model governs whose privileges apply, not where the object can be called.

WHERE THIS CAME FROM

Module 22. Procedures are invoked with CALL as their own statement. No privilege or wrapper changes that.

> A compliance team must let analysts archive records into a protected audit table. Analysts must not be able to write to that table directly, or read it. A stored procedure has been written to perform the archive correctly.
>
> The procedure runs fine for its owner. When an analyst calls it, it fails with an insufficient privileges error on the audit table.

What is wrong?

A. Analysts need INSERT on the audit table. There is no way to avoid granting it.

B. The procedure must be recreated as a secure UDF, which masks the underlying object.

C. The procedure was created with EXECUTE AS CALLER. Owner's rights runs with the owner's privileges instead.

D. Stored procedures cannot write to tables the caller lacks access to, under either rights model.

◆ PAUSE AND DECIDE

What is wrong?

A. Grant INSERT to analysts.

Defeats the requirement. They would then be able to write to the audit table directly, which is what must be prevented.

B. Recreate as a secure UDF.

Secure UDFs hide a definition. They do not elevate privileges, and a UDF cannot perform the DML anyway.

C. It uses caller's rights. Owner's rights delegates the privilege.

Owner's rights runs with the owner's privileges, which is exactly how an operation is delegated without granting the underlying access.

D. Neither rights model permits it.

Owner's rights exists precisely to permit it. It is the documented delegation mechanism.

WHERE THIS CAME FROM

Modules 8 and 22. Owner's rights is the default, so somebody explicitly chose caller's rights here.

> A malformed analytical query was submitted on a Friday evening and nobody noticed. It ran continuously until Sunday, consuming credits the whole time on a Large warehouse, before finally being cancelled by hand.
>
> The account has no resource monitor on that warehouse. A team lead asks what would have stopped it automatically.

What is the most direct control?

A AUTO_SUSPEND. A shorter setting would have suspended the warehouse mid-query.

B STATEMENT_TIMEOUT_IN_SECONDS. Its default is 172800 seconds, which is 48 hours.

C STATEMENT_QUEUED_TIMEOUT_IN_SECONDS, which aborts long-running statements.

D Query acceleration, which would have finished the query sooner.

◆ PAUSE AND DECIDE

What is the most direct control?

A. A shorter AUTO_SUSPEND.

Auto-suspend triggers on inactivity. A warehouse running a query is not inactive, so it never fires.

B. STATEMENT_TIMEOUT_IN_SECONDS, default 48 hours.

It aborts a running statement. The 48-hour default is exactly why a runaway query can burn a weekend.

C. STATEMENT_QUEUED_TIMEOUT_IN_SECONDS.

That governs time spent waiting in the queue, not time spent running. Its default is 0, meaning no limit.

D. Query acceleration.

Speeds up eligible queries. A malformed query finishing sooner is not the goal; stopping it is.

WHERE THIS CAME FROM

Module 17. A resource monitor would also have capped the spend, and the question asks for the most direct control on the statement itself.

> A join between a large fact table and a dimension table is slow. Query Profile shows no spilling, partitions scanned at 4% of total, and rows out roughly equal to rows in at the join.
>
> The dominant category in the execution time breakdown is Synchronization, at 58%.

What does that most likely indicate?

A An exploding join. Synchronization time is how row multiplication appears in the profile.

B Inefficient pruning. 4% is too low and the workers are waiting for data.

C The result cache is being populated, which blocks the workers until it completes.

D Data skew. Parallel workers are waiting on one overloaded colleague.

◆ PAUSE AND DECIDE

What does that most likely indicate?

A An exploding join.

That shows as rows out far exceeding rows in. The scenario says they are roughly equal, which rules it out.

B Inefficient pruning at 4%.

Inverts the ratio. 4% of partitions scanned is excellent pruning; low is good.

C The result cache being populated.

Populating the cache is not an operator and does not block workers. It is not in the time breakdown at all.

D Data skew. Workers waiting on one overloaded colleague.

High Synchronization is the documented fingerprint of skew: parallel workers idle while one finishes.

WHERE THIS CAME FROM

Module 19. The scenario rules out the other three documented problems one by one, leaving the category itself as the answer.

A team needs a function that takes a customer id and returns every address that customer has ever used, as multiple rows, so it can be joined into other queries.

They have written a scalar Python UDF that returns an ARRAY of addresses, and are now flattening the array in every query that calls it.

What would be cleaner?

A. A stored procedure returning tabular data, called with CALL before each query.

B. A UDTF, which is a table function returning a set of rows rather than one value.

C. A vectorized Python UDF, which returns batches of rows instead of single values.

D. A secure UDF, which permits multiple return values when the definition is hidden.

◆ PAUSE AND DECIDE

What would be cleaner?

A. A tabular stored procedure.
It can be called in a FROM clause, and it still needs a CALL and cannot be joined per row like a table function.

B. A UDTF.
A table function returns a set of rows per input, which is exactly the requirement. It joins in naturally with LATERAL.

C. A vectorized Python UDF.
Vectorization is about processing input in batches for speed. It still returns one value per row.

D. A secure UDF.
Secure hides the definition. It has no effect on how many values a function returns.

WHERE THIS CAME FROM

Modules 21 and 22. The array-then-flatten pattern is the signal: it is a UDTF written the long way round.

> A team on Standard Edition has a 20 TB table queried with selective range filters on an order_date column. Query Profile shows partitions scanned close to partitions total.
>
> An architect recommends enabling the Search Optimization Service, and separately suggests a materialized view over the filtered result.

What should actually be done?

A Enable search optimization as recommended. Poor pruning is exactly what it fixes.

B Build the materialized view. It precomputes the filtered result and avoids the scan.

C Upgrade to Enterprise, then enable both, since range filters need search optimization.

D Add a clustering key on order_date. Both recommendations require Enterprise Edition.

◆ PAUSE AND DECIDE

What should actually be done?

A Enable search optimization.

Enterprise only, and it targets point lookups. These are range filters, which is clustering's shape.

B Build the materialized view.

Also Enterprise only, and a view filtered to one range answers nothing outside it.

C Upgrade and enable both.

Spends money to reach a worse answer. Even on Enterprise, range filters point at clustering rather than search optimization.

D Clustering key on order_date. The others need Enterprise.

Standard Edition, and a date column with range filters is the textbook clustering case. It fixes the pruning directly.

WHERE THIS CAME FROM

Modules 2, 3 and 20. Two independent reasons for D: the edition, and the shape of the query. Either alone would decide it.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Procedure value needed inside a SELECT	Module 22 - UDF or stored procedure
2. Analyst cannot write to the audit table	Modules 8 and 22 - owner's rights
3. Query ran all weekend	Module 17 - STATEMENT_TIMEOUT_IN_SECONDS
4. High Synchronization time	Module 19 - the execution time breakdown
5. Array returned then flattened every time	Modules 21 and 22 - UDTFs
6. Standard Edition and poor pruning	Modules 2, 3 and 20 - the tiebreaker table

ON THE EXAM

Questions 2, 5 and 6 span more than one module. Those are worth re-reading even if you got them right.

Domain 4 in five sentences

- **Scale up for one slow query, scale out for many queued ones.** One query runs in one cluster.
- **Read the profile before spending money.** Expensive nodes, partitions scanned, bytes spilled, rows in and out.
- **Clustering for ranges, search optimization for needles, MVs for zero lag, QAS for occasional giants.**
- **Defaults nobody chose:** 48-hour statement timeout, OUTER FALSE, owner's rights, memory that never grows.
- **A UDF returns a value into a query. A procedure performs work.**

ON THE EXAM

Next: Domain 5. Data collaboration at 10%, beginning with Time Travel, Fail-safe and zero-copy cloning.